

ENSF 380

TOPIC 12: STRINGS II

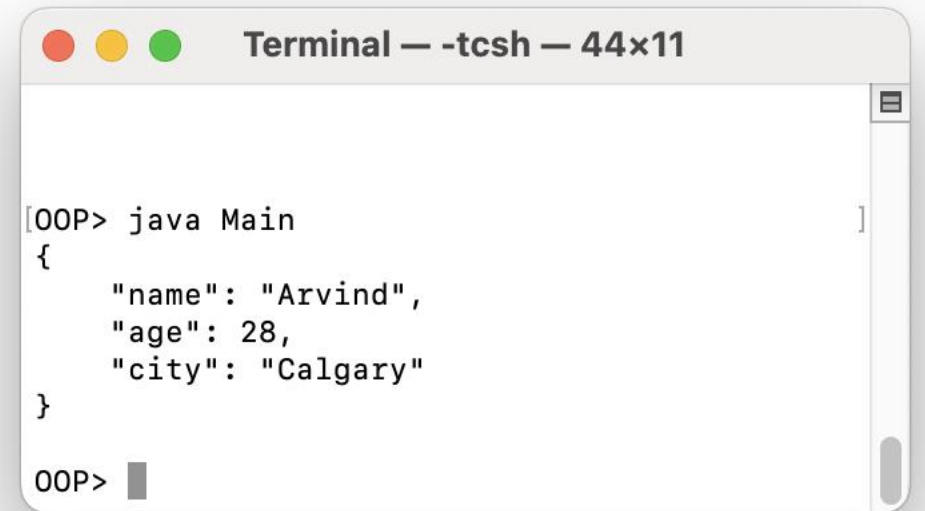
Escape for Literals

`\` is an escape character

- Use it whenever you want the next character to not be interpreted as part of the programming language
- `String myString = "She said, "Hi!"";` is incorrect
- `String myString = "She said, \"Hi!\"";` is correct
- The backslash itself must be escaped to be included literally (`\\`)

Text Blocks

```
public class Main {  
    public static void main(String[] args) {  
  
        String json = ""  
            {  
                "name": "Arvind",  
                "age": 28,  
                "city": "Calgary"  
            }  
        "";  
  
        System.out.println(json);  
    }  
}
```

A terminal window titled "Terminal — -tcsh — 44x11" with standard macOS window controls (red, yellow, green buttons). The terminal shows the command `[OOP> java Main]` being executed. The output is a JSON object: `{ "name": "Arvind", "age": 28, "city": "Calgary" }`. The prompt `OOP>` is visible at the bottom with a cursor.

```
Terminal — -tcsh — 44x11  
[OOP> java Main  
{  
    "name": "Arvind",  
    "age": 28,  
    "city": "Calgary"  
}  
OOP>
```

Regular Expressions

A regular expression (regex) is a pattern for searching and replacing within Strings

Examples:

Explanation	Regex	Example match	Example not match	Example usage
Find a whitespace character	\s	a b c d e f g	abdefghijkl	Remove trailing whitespace
Find a digit	\d	403.829 1968	This sentence has zero numerals in it.	Verify a phone number has the right number of digits

Important: In both a string and regex, \ means "escape" – ignore the special meaning of the character and treat it literally. If you wanted to see the literal characters "\d" in a regex and not have it interpreted as a digit, you'd need to write "\\d"

Example Regular Expressions

Example task: Remove trailing whitespace

- The end of a line: `$`
- Any whitespace: `\s`
- One or more occurrences: `+`
- Match one or more occurrences of whitespace at the end of a line: `\s+$`

Example task: Check if a phone number has 9 digits. Digits must be in the pattern of 3, 3, 4 and can be separated with whitespace, a dash, and/or a period. No other characters are allowed.

- Start of the line: `^`
- A digit: `\d`
- X number of times: `{x}`
- Between X and Z number of times, inclusive: `{x,z}`
- Any character in a list: `[]`
- Zero or more occurrences: `*`
- Match the requested pattern: `^\d{3}[\s.-]{0,1}\d{3}[\s.-]{0,1}\d{4}$`

Regular Expressions in Java

Uses `java.util.regex` package

Creates a search pattern for matching

Pattern class defines the search pattern

- `Pattern.compile()`
- Flags are used to modify the search parameters (e.g. case sensitivity, ignore special characters, etc.)

Matcher class is used to search for the pattern match

Find a Match

02_FindMatches

...

```
import java.util.regex.*;

public class JavaStrings {

    public static void main(String[] args) {

        Pattern myPattern = Pattern.compile("Horses", Pattern.CASE_INSENSITIVE);
        Matcher myMatcher = myPattern.matcher("Horses are mammals. Seahorses are not.");
        boolean isMatch = myMatcher.find();
        System.out.println(isMatch);
    }
}
```

true

Counting Matches

03_CountMatches

```
import java.util.regex.*;

public class JavaStrings {

    public static void main(String[] args) {

        Pattern myPattern = Pattern.compile("Horses", Pattern.CASE_INSENSITIVE);
        Matcher myMatcher = myPattern.matcher("Horses are mammals. Seahorses are not.");
        int matchCount = 0;
        while (myMatcher.find())
            matchCount++;
        System.out.println(matchCount);
    }
}
```

2

Exercise 12.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Splitting Strings

Use delimiters to break Strings into tokens

- `split()`
- First argument indicates what to use to divide the string into tokens – can also be a regular expression
- Second argument is how many tokens to create (0 = as many as you can)

...

```
public static void main(String[] args) {  
  
    String theString = "Elephants are mammals.";  
    String[] tokenizedArray = theString.split(" ", 0);  
    System.out.println("Number of tokens: " + tokenizedArray.length);  
    for (String value: tokenizedArray) {  
        System.out.println(value);  
    }  
}
```

Splitting Strings

Use delimiters to break Strings into tokens

- `split()`
- First argument indicates what to use to divide the string into tokens – can also be a regular expression
- Second argument is how many tokens to create (0 = as many as you can)

...

```
public static void main(String[] args) {  
  
    String theString = "Elephants are mammals."  
    String[] tokenizedArray = theString.split(" ", 0);  
    System.out.println("Number of tokens: " + tokenizedArray.length);  
    for (String value: tokenizedArray) {  
        System.out.println(value);  
    }  
}
```

```
Number of tokens: 3  
Elephants  
are  
mammals.
```

Example Regular Expressions in Java

Example task: Remove trailing whitespace

- The end of a line: \$
- Any whitespace: \s
- One or more occurrences: +
- Match one or more occurrences of whitespace at the end of a line: \s+\$

```
public static String removeTrailingWhitespace(String myLine) {  
    myLine = myLine.replaceAll("\\s+$", "");  
    return myLine;  
}
```

Important: In older versions of Java, regex patterns are Strings. If we want "\s" (match a whitespace) in our pattern, we need to write "\\s" in the String. The first \ tells the String that the second \ is taken literally in the String, and thus the regex receives "\s". To get a literal "\s" in the regex, we'd have to write "\\s".

Example Regular Expressions in Java

Example task: Check if a phone number has 9 digits. Digits must be in the pattern of 3, 3, 4 and can be separated with whitespace, a dash, and/or a period. No other characters are allowed.

- Start of the line: ^
- A digit: \d
- X number of times: {x}
- Between X and Z number of times, inclusive: {x, z}
- Any character in a list: []
- Zero or more occurrences: *
- Match the requested pattern: ^\d{3}[\s.-]{0,1}\d{3}[\s.-]{0,1}\d{4}\$

```
public static boolean isPhoneNumber(String inputNumber) {  
    Pattern phonePat = Pattern.compile(  
        "^\\d{3}[\\s.-]{0,1}\\d{3}[\\s.-]{0,1}\\d{4}$");  
    Matcher myMatcher = phonePat.matcher(inputNumber);  
    return myMatcher.find();  
}
```

```
1231231234: true  
123 123-1234: true  
123.123.1234: true  
01231231234: false  
123.123 _1234: false
```

Example Regular Expressions in Java

Example task: Check if a phone number has 9 digits. Digits must be in the pattern separated with whitespace, a dash, and/or a period. No other characters are allowed.

- Start of the line: `^`
- A digit: `\d`
- X number of times: `{x}`
- Between X and Z number of times, inclusive: `{x,z}`
- Any character in a list: `[]`
- Zero or more occurrences: `*`
- Match the requested pattern: `^\d{3}[\s.-]{0,1}\d{3}[\s.-]{0,1}\d{4}$`

Important: JDK 15 introduced text blocks which allow you to ignore the need to escape regular expressions in strings. Make sure you consider what version of java you are targeting when considering if you need to use escapes or not.

```
public static boolean isPhoneNumber(String inputNumber) {  
    Pattern phonePat = Pattern.compile(  
        "^\\d{3}[\\s.-]{0,1}\\d{3}[\\s.-]{0,1}\\d{4}$");  
    Matcher myMatcher = phonePat.matcher(inputNumber);  
    return myMatcher.find();  
}
```

```
1231231234: true  
123 123-1234: true  
123.123.1234: true  
01231231234: false  
123.123 _1234: false
```

Exercise 12.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Exercise 12.3

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Capturing in Regular Expressions

```
public static String normalizePhoneNumber(String inputPhone) {
    // Define the regex pattern
    String phoneRegex = "^((\\d{3})[\\s.-]?)(\\d{3})[\\s.-]?(\\d{4})$";
    // Define the replacement string
    String replacement = "$1-$2-$3";

    // Use String.replaceAll to replace the text.
    return inputPhone.replaceAll(phoneRegex, replacement);
}

public static void main(String[] args) {
    String inputPhone1 = "123 4567890";
    String inputPhone2 = "123-456_7890";
    String inputPhone3 = "123.456-7890";
    String inputPhone4 = "# 123 456 7890";
}
```

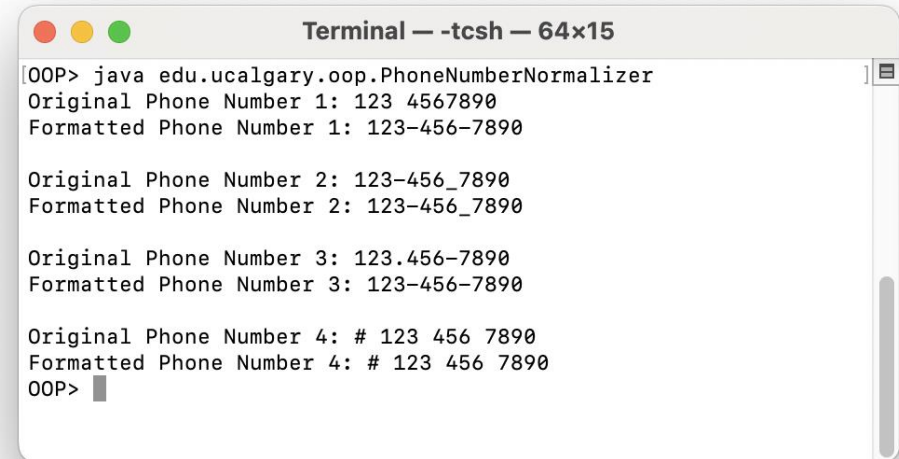
Capturing in Regular Expressions

05_Regex

```
public static String normalizePhoneNumber(String
    // Define the regex pattern
    String phoneRegex = "^((\\d{3})[\\s.-]?)(\\d{3})"
    // Define the replacement string
    String replacement = "$1-$2-$3";

    // Use String.replaceAll to replace the text
    return inputPhone.replaceAll(phoneRegex, replacement);
}
```

```
public static void main(String[] args) {
    String inputPhone1 = "123 4567890";
    String inputPhone2 = "123-456_7890";
    String inputPhone3 = "123.456-7890";
    String inputPhone4 = "# 123 456 7890";
}
```



Terminal — -tcsh — 64x15

```
OOP> java edu.ucalgary.oop.PhoneNumberNormalizer
Original Phone Number 1: 123 4567890
Formatted Phone Number 1: 123-456-7890

Original Phone Number 2: 123-456_7890
Formatted Phone Number 2: 123-456_7890

Original Phone Number 3: 123.456-7890
Formatted Phone Number 3: 123-456-7890

Original Phone Number 4: # 123 456 7890
Formatted Phone Number 4: # 123 456 7890
OOP>
```

Formatted Output

06_FormattedOutput

Standard interpolation

```
+ System.out.print("My cat is " + ageCat +  
    " years old.\n");
```

```
+ String dog = "My dog is " + ageDog + "  
years old.";  
System.out.println(dog);
```

Formatted output

```
+ System.out.printf("My cat is %d years  
old.\n", ageCat);
```

```
+ dog = String.format("My dog is %d years  
old.", ageDog);  
System.out.println(dog);
```

Formatted Output

Important: Later versions of Java have an additional approach for formatting Strings which will work not work in earlier versions of Java.

```
// JDK 11  
dog = String.format("My dog is %d years old.", ageDog);
```

```
// JDK 13  
dog = "My dog is %d years old.".formatted(ageDog);
```

When to Use Formatted Output

Something about the String is unknown.

English	German
Today is January 5th.	Heute ist der 5. Januar.

Something about the variable is unknown.

Actual value	Desired output
85.3	85.3% of the class got an 'A' on this exercise.
56.03498	56.0% of the class got an 'A' on this exercise.

When to Use Formatted Output

06_FormattedOutput

Standard interpolation

```
+ public static String withoutFormatting(String  
before, String after, int day) {  
    return before + day + after;  
}
```

```
...  
  
String englishBefore = "Today is January ";  
String englishAfter = "th.";  
String deutschBefore = "Heute ist der ";  
String deutschAfter = ". Januar";  
System.out.println(withoutFormatting(  
+ englishBefore, englishAfter, 5));
```

String formatting

```
+ public static String withFormatting(String  
sentence, int day) {  
    return String.format(sentence, day);  
}
```

```
...  
  
String english = "Today is January %dth.";  
String deutsch = "Heute ist der %d. Januar.";  
System.out.println(withFormatting(english, 5));
```

To learn more about internationalization standards, look up the ICU (International Components for Unicode) and the MessageFormat class.

String formatting

+

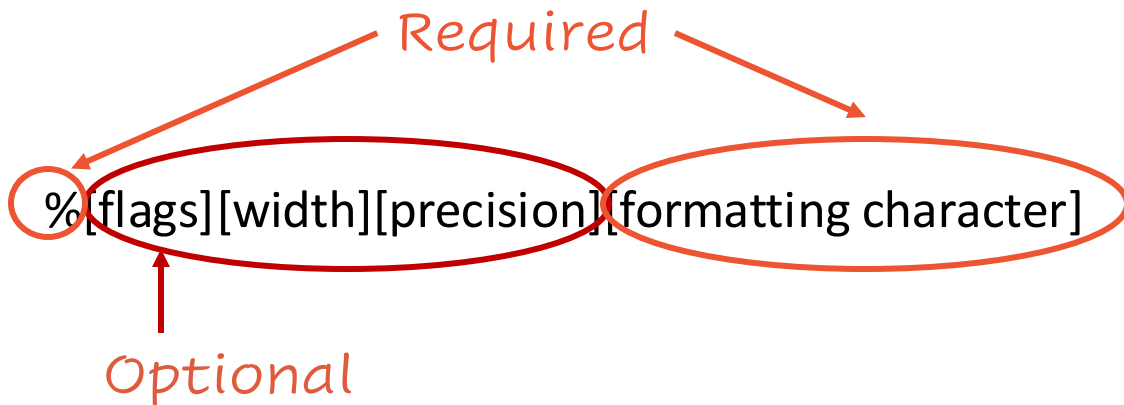
```
public static String withFormatting(String  
sentence, int day) {  
    return String.format(sentence, day);  
}
```

...

```
String english = "Today is January %dth.";  
String deutsch = "Heute ist der %d. Januar.";  
System.out.println(withFormatting(english, 5));
```

Formatted Output

`%[flags][width][precision][formatting character]`



Formatting character	Use
<code>%s</code>	String
<code>%d</code>	Integers
<code>%f</code>	Floating point numbers

```
System.out.printf("The number is %.1f\n", 5.51);
```

Tip: Read about
the syntax for
`printf()`

The number is 5.5

Summary: Knowledge

When to use regular expressions

When to use a tokenizer for parsing

When to use formatted output

Summary: Skills

Define a regular expression

Break a string into tokens

Use String formatting to interpolate variables